

Summary

This chapter covered some classic recursive algorithms. For each, we asked the three important questions you should always ask when designing your own recursive functions: What is the base case? What arguments are passed to the recursive function call? How do these arguments become closer to the base case? If they don't, your function will continue to recurse until it causes a stack overflow.

The summation, string reversing, and palindrome detection recursive functions could have easily been implemented with a simple loop. The key giveaway is that they all make a single pass through the data given to them with no backtracking. As explained in Chapter 2, recursive algorithms are especially suited to problems that involve a tree-like structure and require backtracking.

The tree-like structures for solving the Tower of Hanoi puzzle suggest that it involves backtracking, as the program execution runs from top to bottom, left to right, in the tree. This makes it a prime candidate for recursion, especially since the solution requires two recursive calls of smaller towers.

The flood fill algorithm is directly applicable to graphics and drawing programs, as well as other algorithms to detect the shape of contiguous areas. If you've used the paint-bucket tool in a graphics program, you've likely used a version of the flood fill algorithm.

The Ackermann function is an excellent example of how quickly a recursive function can grow as its inputs increase. While it doesn't have many practical applications in day-to-day programming, no discussion about recursion would be complete without it. But as recursive as it is, like all recursive functions it can be implemented iteratively with a loop and a stack.